

QASPER RAG Project Walkthrough

CS505 NLP Final Project Team

April 16, 2026

1 Project Overview

This project builds a **local, reproducible retrieval-augmented question answering system** for scientific papers using the **QASPER** dataset. The task is: given a paper and a question about that paper, retrieve the most relevant evidence passages and generate a grounded answer. In the citation-forcing setting, the system must also cite the retrieved evidence it claims to rely on.

The system solves three linked subproblems:

1. **Preprocessing and chunking:** convert raw scientific papers into stable, retrievable passages.
2. **Retrieval:** find the most relevant chunks for each question.
3. **Generation and grounding:** generate an answer, optionally with citations, and evaluate whether the answer is correct and whether the citations are actually supported.

At a high level, the project:

- preprocesses QASPER into canonical shared artifacts,
- supports fixed, sliding-window, and section-aware chunking,
- evaluates BM25, dense retrieval, hybrid retrieval, and reranking,
- generates answers with local models such as Phi and Qwen,
- evaluates answer quality, semantic similarity, grounding, and citation quality,
- produces report-ready tables, analyses, and human-evaluation artifacts.

2 Repository Structure

2.1 Top-Level Layout

NLP_research_project/	
src/qasper_rag/	core library code
scripts/	entrypoints, batch jobs, backfills, analysis

tests/	unit and regression tests
reports/	proposal, midway report, status report, artifacts
local_scc_results/	archived SCC outputs copied locally
README.md	repo usage guide
setup_env.sh	environment setup
environment.yml	conda environment spec

2.2 What Each Folder Is For

- **src/qasper_rag/**: the actual implementation of loading, chunking, retrieval, generation, and evaluation.
- **scripts/**: operational entrypoints used to build data, run experiments, submit SCC jobs, backfill metrics, analyze errors, and prepare human evaluation.
- **tests/**: correctness checks for the main components and regression tests for tricky bugs.
- **reports/**: LaTeX papers and supporting artifacts used for the proposal, midway report, and internal status tracking.
- **local_scc_results/**: a persistent local mirror of important SCC outputs so experiment JSONs are not lost.

3 File-by-File Guide

3.1 Core Library Files

File	Role	Key functions/classes	Connections
<code>config.py</code>	Central path and layout resolver.	<code>ChunkingSpec</code> , <code>ProjectPaths</code> , <code>get_default_project_paths()</code> , <code>ensure_project_layout()</code> .	Used by almost every script to resolve SCC shared paths, per-user run paths, and local fallbacks.
<code>schema.py</code>	Normalized in-memory representation of raw QASPER.	<code>QasperPaper</code> , <code>QasperQuestion</code> , <code>QasperAnswer</code> , <code>Section</code> , <code>TextUnit</code> , <code>Chunk</code> .	Produced by <code>loader.py</code> ; consumed by <code>chunking.py</code> and <code>processing.py</code> .
<code>loader.py</code>	Loads raw QASPER files or Hugging Face dataset records.	<code>load_qasper_json()</code> , <code>load_qasper_huggingface()</code> , <code>resolve_qasper_split_path()</code> , <code>parse_qasper_paper()</code> .	Feeds normalized paper/question/answer objects into <code>processing.py</code> .
<code>chunking.py</code>	Implements all chunking strategies.	<code>BaseChunker</code> , <code>FixedChunker</code> , <code>SlidingWindowChunker</code> , <code>SectionAwareChunker</code> , token-window helpers.	Called by <code>processing.py</code> to build retrievable chunk corpora.
<code>processing.py</code>	Builds canonical processed artifacts from raw QASPER.	<code>build_processed_split()</code> , <code>process_qasper_split_file()</code> , record serializers and JSONL writers.	Bridges raw input and the shared <code>processed/</code> interface used by the rest of the pipeline.
<code>artifacts.py</code>	Loads processed artifacts back from disk.	<code>ProcessedQuestion</code> , <code>ProcessedChunk</code> , <code>load_processed_questions()</code> , <code>load_processed_chunks()</code> , <code>group_chunks_by_paper()</code> .	Used by retrieval and generation runners.

File	Role	Key functions/classes	Connections
retrieval.py	Implements the retrieval stack.	BM25Retriever, DenseRetriever, HybridRetriever, CrossEncoderReranker, RerankRetriever, NullRetriever, RandomRetriever, build_retriever().	Used by both retrieval-only and retrieval+generation experiments.
retrieval_eval.py	Evaluates retrieval quality against QASPER evidence.	QuestionRetrievalMetrics, evaluate_ranked_chunks(), aggregate_question_metrics(), evidence matching helpers.	Used by evaluate_qasper_retrieval.py.
generation.py	Builds prompts, runs local generation, parses citations, normalizes outputs.	PromptPackage, GenerationPrediction, HuggingFaceGenerator, build_prompt(), prepare_prompt_chunks(), normalize_generation_answer(), build_generation_prediction().	Central generation logic used by the generation runner.
generation_eval.py	Evaluates answer quality and grounding.	QuestionGenerationMetrics, token_f1_score(), collect_gold_answers(), evaluate_generation_prediction(), aggregate_generation_metrics().	Used during generation experiments and in metric backfills.
nli_eval.py	Checks whether cited claims are supported by evidence.	HuggingFaceEntailmentScorer, CitedClaim, extract_cited_claims(), evaluate_cited_claims().	Used for NLI citation backfill and citation verification.
citation_verifier.py	New verifier gate for citation-forcing outputs.	verify_citation_prediction().	Called inside the generation runner to keep only supported cited claims or abstain to unanswerable.
__init__.py	Package export surface.	Re-exports the main classes and functions.	Makes imports cleaner in scripts and tests.

3.2 Important Script Files

File	Role	What it is used for
build_qasper_processed.py	Processed data builder	Generates canonical papers.jsonl, questions.jsonl, chunks.*.jsonl, and manifest.json.
evaluate_qasper_retrieval.py	Retrieval runner	Runs retrieval-only experiments and writes retrieval JSON outputs.
evaluate_qasper_generation.py	Generation runner	Runs retrieval + generation + evaluation end-to-end; now also supports citation verification.
qsub_qasper_retrieval.sh	SCC batch wrapper	Runs retrieval jobs on SCC.
qsub_qasper_generation.sh	SCC batch wrapper	Runs generation jobs on SCC with model, retrieval, and citation-verifier flags.
submit_proposal_ablation.sh	Proposal batch chain	Submits the Phi A–E experiment chain.
submit_proposal_baselines.sh	Baseline batch chain	Submits parametric-only, BM25-only, and random-retrieval baselines.
submit_full_validation_phi.sh	Full-validation runner	Submits the strongest Phi settings on full validation.
submit_full_validation_ab.sh	Full-validation runner	Submits full-validation A and B to complete the proposal matrix.
backfill_bertscore.py	Metric backfill	Adds BERTScore to existing generation JSONs without rerunning generation.
backfill_nli_citation.py	Metric backfill	Adds NLI citation metrics to existing generation JSONs.
analyze_generation_errors.py	Error analysis	Groups generation failures into categories such as retrieval misses and unsupported citations.

File	Role	What it is used for
<code>compare_generation_runs.py</code>	Run comparison	Compares two saved generation runs on overlapping questions.
<code>summarize_generation_runs.py</code>	Result packaging	Builds markdown tables from saved result JSONs.
<code>prepare_human_eval_sample.py</code>	Human-eval preparation	Builds an enriched reviewer sample with gold answers, evidence, cited chunk text, and NLI claim hints.
<code>create_human_eval_template.py</code>	Human-eval preparation	Turns a sample CSV into fillable reviewer templates.
<code>aggregate_human_eval.py</code>	Human-eval aggregation	Merges completed reviewer CSVs into summary tables and JSON.
<code>preview_qasper_chunking.py</code>	Debugging/sanity check	Lets the team inspect chunking behavior for a small paper subset.
<code>prepare_shared_qasper.sh</code>	Dataset setup	Downloads and extracts public QASPER archives into the shared SCC directory.
<code>pull_scc_results.sh</code>	Result archival	Safely copies SCC <code>runs/</code> and <code>logs/</code> locally without deleting the remote copies.
<code>show_project_paths.py</code>	Debugging	Prints the resolved SCC/local path layout.
<code>install_bertscore_overlay.sh</code>	SCC support utility	Installs a project-local overlay for BERTScore dependencies.
<code>install_qwen3_overlay.sh</code>	SCC support utility	Installs a project-local transformers overlay needed for Qwen3.

3.3 Reports, Tests, and Artifacts

- `reports/proposal.tex`: original project proposal.
- `reports/midway_report.tex`: ACL-formatted midway report.
- `reports/status_report.tex`: internal running status report with the latest experiment state.
- `reports/references.bib`: shared bibliography.
- `reports/artifacts/final_validation_results.md`: final consolidated full-validation generation table.
- `reports/artifacts/citation_tuning_results.md`: tracked citation-tuning sample results.
- `reports/artifacts/human_eval_*`: reviewer sample, templates, instructions, and aggregated human-evaluation summary.
- `tests/test_*.py`: unit and regression tests covering loading, chunking, retrieval, generation, NLI, configuration, and citation verification.

4 Overall Workflow

4.1 From Input to Output

1. **Prepare raw QASPER data.** The project downloads and extracts the train/dev/test QASPER files.
2. **Normalize papers and questions.** Raw papers, questions, answers, and evidence are parsed into a consistent internal schema.

3. **Chunk papers.** Each paper is split into retrievable units using fixed-size, sliding-window, or section-aware chunking.
4. **Write canonical processed artifacts.** These become the shared interface for all later experiments.
5. **Run retrieval.** For each question, the system retrieves top chunks using BM25, dense retrieval, hybrid fusion, or reranking.
6. **Build a prompt from retrieved chunks.** Retrieved chunks are compressed into the prompt context and labeled so the model can cite them.
7. **Generate an answer.** The model produces either a baseline answer or a citation-forcing answer.
8. **Normalize and parse the output.** The system extracts citations, cleans the answer, and normalizes cases like `yes`, `no`, and `unanswerable`.
9. **Verify citations (new step).** In verifier mode, cited claims are checked with NLI. Unsupported cited outputs can be collapsed to `unanswerable`.
10. **Evaluate.** Retrieval metrics, token F1, BERTScore, support rate, citation precision, and NLI citation precision are computed.
11. **Archive and summarize.** SCC results are copied locally, summarized into markdown tables, and used for reports and human evaluation.

5 Results and Outputs

5.1 Retrieval Results

Method	Recall@5	MRR@10	Evidence Hit
BM25	0.5769	0.4110	0.7152
Dense	0.5886	0.4820	0.7238
Hybrid	0.6313	0.5097	0.7778
Hybrid + rerank	0.6410	0.5027	0.7702

Table 3: Validation retrieval results on section-aware chunks.

Interpretation:

- Hybrid retrieval is the best balanced retriever.
- Reranking improves recall slightly, but that gain does not automatically translate into the best downstream generation quality.

Cond.	Chunking	Retrieval	Prompt	F1	BERT	Halluc.	Support	NLI Cite
A	fixed	dense	baseline	0.3059	0.8791	0.5134	0.6886	0.0000
B	fixed	hybrid	baseline	0.3063	0.8782	0.5104	0.6716	0.0000
C	section	hybrid	baseline	0.3197	0.8815	0.4527	0.7741	0.0000
D	section	hybrid_rerank	baseline	0.3238	0.8817	0.4478	0.7652	0.0000
E	section	hybrid_rerank	citation	0.3215	0.8819	0.5463	0.5960	0.1416
P	section	none	baseline	0.1350	0.8329	0.8657	0.1343	0.0000
M	section	BM25	baseline	0.3207	0.8813	0.4716	0.7254	0.0000

Table 4: Main full-validation Phi results.

5.2 Full-Validation Generation Results

What these mean:

- **D is the best overall Phi setting.** Section-aware chunking plus hybrid retrieval with reranking and baseline prompting gives the best answer-quality and grounding balance.
- **E is good at producing citations, but not at trustworthy attribution.** It cites very often, but many cited claims are not actually supported.
- **P is much worse.** This shows retrieval is genuinely important; the model alone is not enough.
- **Section-aware chunking matters.** The jump from A/B to C/D shows that structural chunking helps more than simply changing retriever type on fixed chunks.

5.3 Human Evaluation

System	Correctness	Grounding	Citation Quality
Baseline	1.021	1.062	—
Citation	0.875	0.708	0.583

Table 5: Mean human-evaluation scores over 24 questions and 2 reviewers.

Additional human-eval findings:

- 24 questions were reviewed by 2 reviewers.
- Exact preference agreement was 83.3%.
- Majority preference by question was split: 9 baseline, 9 citation, 6 tie.

Why this is significant:

- Automatic metrics say baseline is stronger on correctness and grounding.
- Human judgments agree baseline is generally safer, but citation answers are not uniformly disliked.
- This means citation forcing is not useless, but it is not yet reliable enough to replace baseline as the main answer mode.

6 Important Concepts and Design Decisions

- **Deterministic processed artifacts:** the team uses a shared `processed/` interface instead of rebuilding chunks differently on each machine.
- **Custom chunking instead of LangChain:** this keeps chunk IDs, metadata, and chunk boundaries stable and reproducible.
- **Hybrid retrieval:** combines sparse and dense retrieval using reciprocal-rank fusion.
- **Question-aware context compression:** retrieved chunks are compressed to relevant sentences before prompting, to fit SCC memory limits and reduce irrelevant context.
- **Two answer modes:** baseline prompting prioritizes answer quality, while citation forcing prioritizes explicit attribution.
- **Multi-metric evaluation:** token F1, BERTScore, support rate, hallucination proxy, citation precision, and NLI citation precision each capture different failure modes.
- **Move from prompt-only tuning to verification:** the new citation verifier is a deliberate shift from “tell the model to cite” toward “check whether the cited claim is actually supported.”

7 Limitations and Possible Improvements

7.1 Current Limitations

- Citation forcing still produces many unsupported cited claims.
- Token F1 is harsh and under-rewards semantically correct paraphrases.
- Human evaluation is still small.
- SCC scheduler/auth instability slowed iteration and some runs had to be retried.
- Some optional proposal stretch items remain secondary rather than central.

7.2 Best Improvements

- Run the new verifier-gated citation mode on SCC and compare it directly to the current citation baseline.
- If the verifier helps, run it on full validation.
- Push citation mode toward a more explicit *generate* $\rightarrow$ *verify* $\rightarrow$ *cite* pipeline.
- Add a second faithfulness metric only if it adds signal beyond the current NLI check.
- Keep stretch-model experiments secondary until the final writeup is locked.

8 Simple Layman Explanation

Imagine giving a long scientific paper to a student and asking, “What baseline did they compare against?” or “Did they only test English?” This project builds a system that tries to answer those questions automatically.

It works in three stages:

1. it cuts the paper into smaller pieces,
2. it searches those pieces to find the most relevant ones,
3. it uses a language model to answer the question based only on those pieces.

We tested different ways of cutting the paper up, different ways of searching for the right evidence, and different ways of forcing the model to show where its answer came from.

The main lesson is:

- getting the right evidence matters a lot,
- the best current version answers best when allowed to answer normally,
- forcing the system to always add citations makes it cite more often, but those citations are not always truly trustworthy yet.

So in plain language, the project is a **research-paper question answering assistant** that already works reasonably well, but still needs a stronger evidence-checking step before its citations can be trusted.